

Feed-Forward Neural Network Implementation and Optimization

Comparing gradient descent methods and network architecture on regression and classification tasks

Sander Oppen and Ole Markus Bråthen
University of Oslo

(Dated: November 10, 2025)

In this project, we will focus on the development and implementation of a Feed-Forward Neural Network (FFNN) from scratch in Python, applied to both regression and multiclass classification tasks. For regression, we revisit the one-dimensional Runge function from Project 1, comparing the FFNN's performance against ordinary least squares (OLS), Ridge, and LASSO regression using the mean squared error (MSE) cost function. The network incorporates flexible architectures with variable hidden layers, nodes and activation functions (Sigmoid, ReLU, Leaky ReLU), optimized via gradient descent methods including plain GD, SGD, RMSProp, and ADAM, with optional L1/L2 regularization. For classification, we employ the MNIST dataset, utilizing softmax cross-entropy loss, and evaluating accuracy. Results demonstrate that the FFNN outperforms traditional linear methods on non-linear data, achieving lower MSE in regression and high accuracy (up to 97%) in classification, though it requires careful hyperparameter tuning to avoid overfitting. Comparisons with PyTorch and Autograd implementations validate our code, with our FFNN achieving the same top-end performance.

I. INTRODUCTION

The advent of deep learning has transformed computational approaches to data-driven problems, allowing models to approximate complex functions that traditional methods struggle with, such as linear regression on certain functions.

Building on Project 1, where we applied ordinary least squares (OLS), Ridge, and LASSO regression to datasets like the one-dimensional Runge function, this project shifts focus to neural networks as a more adaptive alternative. Linear models, while efficient and interpretable, often fail to model non-linear functions without high-degree polynomials, leading to overfitting or instability. Feed-forward neural networks (FFNNs), inspired by biological neurons, address this by stacking layers of interconnected nodes with non-linear activations, enabling the modeling of a wider range of problems.

Our core aim is to develop and rigorously test a from-scratch FFNN implementation in Python, using back-propagation for training. For regression, we reuse the Runge function 1 with added noise, comparing MSE outcomes across neural network architectures and different optimizers.

In classification, the MNIST handwritten digits dataset featuring 70,000 grayscale images tests the networks multiclass capabilities.

We explore a wide variety of methods to improve the networks performance, including the impact of activation functions, regularization to combat overfitting, and gradient descent improvements like ADAM for adaptive learning rates.

The implementation is validated and compared against established software libraries like Autograd for checking the correctness of gradients, and PyTorch for benchmarking the networks performance.

This report proceeds as follows: Section 2 outlines theory and the methods we will use in the project, it de-

scribes the code architecture, implementation, and our use of AI tools. Section 3 presents the regression and classification experiments and results. We also discuss the model performance and compare to other implementations and previous projects when applicable. Finally, in Section 4 we present the conclusions from the experiments, as well as future avenues for exploration.

All implementations of the methods and experiments can be found in the project repository.

II. METHODS

A. Data

1. Runge Function

For regression with neural networks, we will use the Runge function is defined as

$$f(x) = \frac{1}{1 + 25x^2}. \quad (1)$$

The x -values in the training and testing data is given by n equally spaced points on the interval $x \in [0, 1]$. The Runge function in particular is much studied in the field of regression and polynomial interpolation, as it gives rise to the Runge phenomenon, where high polynomial degrees can perform poorly.[1]

2. MNIST handwritten digits

The modified National Institute of Standards and Technology (MNIST) dataset is a collection of 70,000 labeled handwritten digits. 60,000 of these are used for training the neural networks, and 10,000 are used as a testing set. All images are black and white, normalized

to a size of $28 \times 28 = 784$ pixels, where each image is represented as a vector of length 784. Due to the datasets simplicity and availability, the existing literature on using neural network to classify the dataset is extensive.[2]

a. One-hot encoding

The labels for each image is given in the dataset as an integer value between 0 and 9. A label k is encoded as a length 10 array of zeros, where the k -th element is set to 1. This is done to make the model more generalizable and easier to work with.

B. Cost/loss functions

1. Mean Squared Error (MSE)

Mean Squared Error (MSE) is a commonly used loss function for regression tasks in neural networks, as it penalizes large errors quadratically and is differentiable, facilitating gradient-based optimization. For a dataset with n samples, where y_i are the true target values and $\hat{y}_i = f(x_i, \theta)$ are the model predictions, the Mean Squared Error is defined as

$$C_{\text{MSE}}(\hat{y}) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2. \quad (2)$$

Differentiating, we get the gradient

$$\nabla C_{\text{MSE}}(\hat{y}) = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) = \frac{2}{n} \sum_{i=1}^n (\hat{y}_i - y_i). \quad (3)$$

Adding regularization terms to MSE can help prevent overfitting by penalizing large model parameters, promoting simpler models that generalize better.

a. L2 regularization

L2 regularization, also known as ridge regularization, adds a penalty proportional to the square of the parameter magnitudes, which encourages small but non-zero weights and is effective in handling multicollinearity in the features. The L2 cost function is given by

$$C_{\text{MSE}}^{L2}(\hat{y}) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \|\mathbf{w}\|_2^2, \quad (4)$$

with gradient

$$\nabla C_{\text{MSE}}^{L2}(\hat{y}) = \frac{2}{n} \sum_{i=1}^n (\hat{y}_i - y_i) + 2\lambda \mathbf{w}. \quad (5)$$

b. L1 regularization

L1 regularization, or lasso regularization, promotes sparsity by driving some parameters to exactly zero, effectively performing feature selection during training.

$$C_{\text{MSE}}^{L1} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \|\mathbf{w}\|_1, \quad (6)$$

with gradient

$$\nabla C_{\text{MSE}}^{L1} = \frac{2}{n} \sum_{i=1}^n (\hat{y}_i - y_i) + \lambda \text{sign}(\mathbf{w}). \quad (7)$$

2. Binary Cross-Entropy (Log Loss)

Binary Cross-Entropy is the preferred loss function for binary classification problems, as it measures the divergence between predicted probabilities and true labels, encouraging confident predictions. For binary classification, where the target $y_i \in \{0, 1\}$ and the predicted probability $\hat{y}_i = p(y_i = 1|x_i)$ (often from a sigmoid activation), the Binary Cross-Entropy loss is

$$C_{\text{BCE}} = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]. \quad (8)$$

The gradient with respect to the predictions is given by

$$\frac{\partial C_{\text{BCE}}}{\partial \hat{y}_i} = -\frac{1}{n} \left(\frac{y_i}{\hat{y}_i} - \frac{1 - y_i}{1 - \hat{y}_i} \right). \quad (9)$$

If the model output comes from a sigmoid activation $\hat{y}_i = \sigma(z_i)$, this simplifies to

$$\frac{\partial C_{\text{BCE}}}{\partial z_i} = \frac{1}{n} (\hat{y}_i - y_i). \quad (10)$$

a. Regularization

Regularization can be added to BCE to control model complexity, similar to MSE, by incorporating L2 penalties on the weights.

$$C_{\text{BCE}}^{L2} = C_{\text{BCE}} + \lambda \|\mathbf{w}\|_2^2, \quad (11)$$

with gradient

$$\nabla C_{\text{BCE}}^{L2} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) + 2\lambda \mathbf{w}. \quad (12)$$

L1 regularization can also be used with BCE

$$C_{\text{BCE}}^{L1} = C_{\text{BCE}} + \lambda \|\mathbf{w}\|_1 \quad (13)$$

with gradient

$$\nabla C_{\text{BCE}}^{L1} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) + \lambda \text{sign}(\mathbf{w}). \quad (14)$$

The Binary Cross-Entropy will however not be used too much in this project, since the main classification we do is on the MNIST dataset, which is a multiclass problem.

3. Multiclass Cross-Entropy (Softmax Loss)

Multiclass Cross-Entropy extends BCE for problems with more than two classes, combining softmax activation to produce probability distributions over classes. For multiclass classification with K classes, true one-hot encoded labels y_{ik} , and predicted probabilities from the softmax function

$$\hat{y}_{ik} = \frac{e^{z_{ik}}}{\sum_{j=1}^K e^{z_{ij}}}, \quad (15)$$

the loss function is

$$C_{\text{Softmax}} = -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K y_{ik} \log(\hat{y}_{ik}). \quad (16)$$

The gradient with respect to the logits simplifies elegantly, making it computationally efficient for backpropagation in neural networks.

$$\frac{\partial C_{\text{Softmax}}}{\partial z_{ik}} = \frac{1}{n} (\hat{y}_{ik} - y_{ik}). \quad (17)$$

C. Activation functions

We will be using three different activation functions: Sigmoid, ReLU, and the leaky ReLU functions. In addition, we use the linear function $f(z) = z$, with derivative $f'(z) = 1$ as the final layer of regression neural networks.

1. Sigmoid

The sigmoid activation function is given by

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (18)$$

with its derivative given by

$$\sigma'(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = (1 - \sigma(z))\sigma(z). \quad (19)$$

2. ReLU

The rectified linear unit (ReLU) function is given as a piecewise function

$$\text{ReLU}(z) = \begin{cases} z & z > 0 \\ 0 & z \leq 0 \end{cases} \quad (20)$$

with derivative

$$\text{ReLU}'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}. \quad (21)$$

3. Leaky ReLU

The leaky ReLU function is a modification of the ReLU functions which is modified to help avoid the vanishing gradient problem by having a non-zero derivative.[3] For this report, will use $\alpha = 0.01$ in the leaky ReLU.

$$\text{LeakyReLU}(z) = \begin{cases} z & z > 0 \\ \alpha z & z \leq 0 \end{cases} \quad (22)$$

Derivative by piecewise derivation

$$\text{LeakyReLU}'(z) = \begin{cases} 1 & z > 0 \\ \alpha & z \leq 0 \end{cases} \quad (23)$$

D. Neural Network methods

The architecture of the n -layer feed-forward neural networks we are implementing can be described by

- Weight matrices $\{W_i\}_{i \in \{1, \dots, n\}}$
- Bias vectors $\{b_i\}_{i \in \{1, \dots, n\}}$
- Activation functions $\{f_i\}_{i \in \{1, \dots, n\}}$.

A prediction is made with the neural network using a feed forward pass. Putting all training data through a single feed forward pass can give us a score on the networks performance using the cost function, which is then used to calculate the gradient for each layer of the network, which is then used to update each layer.

1. Feed forward

If we let a_i be the output of the i -th layer of the network, then the $i + 1$ -th layer has output

$$\begin{aligned} z_i &= W_i a_i + b_i \\ a_{i+1} &= f_i(z_i). \end{aligned}$$

2. Gradient calculation

To use gradient descent methods to update the layer weights, we first need a method to calculate the gradients for each layer of the neural network. As the gradient for the last layer of the network is relatively easy to compute, we use that as a starting point and repeatedly use the chain rule to compute the gradients for the earlier layers.

For each layer, we wish to compute $\frac{dC}{dW_i}$ and $\frac{dC}{db_i}$. We begin by finding an expression for the weights. If we express

$$\frac{dC}{dW_i} = \frac{dC}{da_i} \frac{da_i}{dz_i} \frac{dz_i}{dW_i}$$

we can attempt to compute each of the terms individually, beginning from the n -th layer and working towards layer 1. Beginning with $\frac{dC}{da_i}$, we have two cases. Firstly, if $i = n$ (the last layer), then this expression is simply the derivative of the given cost function, $C'(a_n)$. Otherwise, we can write

$$\frac{dC}{da_i} = \frac{dC}{dz_{i+1}} \frac{dz_{i+1}}{da_i}.$$

The first term we have already calculated in the previous iteration. The second term, we use that $z_{i+1} = W_{i+1}a_i + b_{i+1}$, so

$$\frac{dz_{i+1}}{da_i} = W_{i+1}.$$

The term $\frac{da_i}{dz_i}$ is easy to compute, as $a_i = f_i(z_i)$, so

$$\frac{da_i}{dz_i} = f'_i(z_i).$$

Finally, we have from the definition of z_i ,

$$\frac{dz_i}{dW_i} = a_{i-1}.$$

The calculation for the bias derivative is similar, only the last term is changed, with

$$\frac{dz_i}{db_i} = 1.$$

Thus, we have a formula for calculating the derivatives for a given layer i :

$$\begin{aligned} \frac{dC}{dz_i} &= \left(\frac{dC}{dz_{i+1}} W_{i+1} \right) f'_i(z_i) \\ \frac{dC}{dW_i} &= \frac{dC}{dz_i} \otimes a_{i-1} \\ \frac{dC}{db_i} &= \frac{dC}{dz_i} \end{aligned}$$

The $\otimes$ denotes the outer product between the vectors.

E. Gradient descent methods

Gradient descent methods are used for updating the layer weights and biases using the computed gradient of the neural network. We will use three different gradient descent algorithms, plain (normal) gradient descent, Root Mean Square Propagation (RMSProp), and Adaptive Moment Estimation (ADAM). Each algorithm will produce a change for the weights and biases ΔW and Δb . For simplicity, we will only provide formulas for ΔW , but the bias gradients are computed in an identical matter.

1. Gradient descent

Gradient descent takes a learning rate η as input. Then, for a given gradient $\mathbf{g}$ it calculates the change

$$\Delta W = -\eta \mathbf{g}$$

2. RMSProp

RMSProp uses a decaying average of the squared gradients to adapt the learning rates as the network weights converge. As input, it takes a learning rate η , and a hyperparameter ρ . It also has an accumulation variable, initially set to $\mathbf{r} = 0$. For a given gradient $\mathbf{g}$, each step of the algorithm computes

$$\begin{aligned} \mathbf{r} &= \rho \mathbf{r} + (1 - \rho) \mathbf{g} \odot \mathbf{g} \\ \Delta W &= -\frac{\eta}{\sqrt{\delta + \mathbf{r}}} \odot \mathbf{g}. \end{aligned}$$

Here $\odot$ describes element-wise multiplication of vectors, and δ is a small constant ($\sim 10^{-6}$) to avoid division by zero.

3. ADAM

ADAM was developed as an attempt to combine the RMSProp algorithm with momentum gradient descent, to achieve faster convergence and to require less tuning. It uses two momentum variables $\mathbf{s} = 0$ and $\mathbf{r} = 0$ for decaying gradients and squared gradients. In addition, some bias correction is computed each iteration, based on the iteration number t . The algorithm takes as input a learning rate η , and hyperparameters ρ_1, ρ_2 for the decay. Each step of the algorithm computes the following

$$\begin{aligned} t &= t + 1 \\ \mathbf{s} &= \rho_1 \mathbf{s} + (1 - \rho_1) \mathbf{g} \\ \mathbf{r} &= \rho_2 \mathbf{r} + (1 - \rho_2) \mathbf{g} \odot \mathbf{g} \\ \hat{\mathbf{s}} &= \frac{\mathbf{s}}{1 - \rho_1^t} \\ \hat{\mathbf{r}} &= \frac{\mathbf{r}}{1 - \rho_2^t} \\ \Delta W &= -\frac{\eta \hat{\mathbf{s}}}{\sqrt{\hat{\mathbf{r}} + \delta}} \end{aligned}$$

4. Stochastic methods

Each of the three previous methods can be implemented as stochastic methods, where for each epoch of the training the data is shuffled and the gradients are computed based not on the entire training data, but rather on smaller minibatches. This method of resampling the training data provides a "free" way to create more training data, as the random sampling of the data will by the central limit theorem give a better representation of the underlying distribution given enough resampling has taken place.

F. Metrics

To measure the accuracy of the regression and classification networks, we need some metrics to quantify the

performance of the network. We use the aforementioned MSE, as well as R^2 and accuracy scores.

1. R^2 -score

The R^2 score used for regression is defined for true values y and predicted values $\hat{y}$ as

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (24)$$

where $\bar{y}$ is the mean of the true values $\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$.

2. Accuracy score

The accuracy score is used for classification problems, and expresses the percentage of the test cases that the model correctly predicted. It is given by

$$\text{Accuracy} = \frac{\sum_{i=1}^n I(t_i = y_i)}{n}. \quad (25)$$

Here I is the indicator function, taking the value 1 if the given expression is true, and 0 otherwise.

G. Implementation

The main neural network is implemented as a Python class `NeuralNetwork`, which initializes layers with random weights and biases using NumPy. The class constructor takes information about the layer shapes, activation functions and their derivatives, cost function, and regularization.

Feed-forward computes activations layer by layer: for input a_{i-1} , $z_i = W_i a_{i-1} + b_i$, $a_i = f_i(z_i)$. Backpropagation calculates gradients starting from the output: $\delta_n = \nabla C \odot f'_n(z_n)$, then $\delta_i = (W_{i+1}^T \delta_{i+1}) \odot f'_i(z_i)$, with weight/bias gradients $\partial C / \partial W_i = \delta_i a_{i-1}^T$ and $\partial C / \partial b_i = \delta_i$ (averaged over batches).

Optimizers (GD, RMSProp, ADAM) update parameters, with stochastic variants shuffling data and using minibatches. Training loops over epochs, computing gradients per sample or batch. Predictions use argmax on softmax outputs. Due to the matrices being implemented as NumPy arrays and no parallelization being used, the performance of the code is extremely slow. (Some networks used over 40 minutes on 15 epochs)

Most derivatives are tested against automatic differentiation using Autograd to check for correctness. The correctness of the neural networks are also verified through the comparison to existing PyTorch implementations of neural networks.

H. Use of AI tools

We have used AI tools to aid in the writing of this project. None of the text is directly generated from the

	MSE	R^2
OLS	0.00442	0.9661
NN ₁	0.00018	0.9986
NN ₂	0.01036	0.9205

Table I. R^2 scores for regression and neural networks

	MSE	R^2
PyNN ₁	0.00038	0.9971
PyNN ₂	0.00045	0.9965

Table II. R^2 scores for regression and neural networks using PyTorch implementation

LLM, but it has been reviewed and improved upon using suggestions from the AI. The AI has also been used for some code generation, code and L^AT_EX debugging. We have tried to not use an excess amount of AI, as the climate impact of large language models is not insignificant. For our AI, we have used the locally hosted GPT UiO, which uses models from OpenAI. We have used this model as it is the most privacy-focused option available. All conversations with the AI can be found in the project repository

III. RESULTS AND DISCUSSION

A. Regression

Our aim is to beat the performance of the previous linear regression methods. From earlier work, we know that for polynomial regression on the Runge function with $n = 50$ points, the best performing regression is a normal OLS regression with a degree 14 polynomial.[4] We trained and tested our neural network on the exact same training and testing data as the OLS regression, using a stochastic RMSProp gradient descent with $\rho = 0.99$, $\eta = 0.01$, Epochs = 5000, and a minibatch size of 5. We train two networks; network NN₁ has one hidden layer with 50 nodes, and network NN₂ has two hidden layers with 100 nodes in each layer, both networks using the sigmoid function between layers.

From the results shown in table I, it is clear that even a simple neural network far outperforms the polynomial regression, as the one-layer network has an MSE 44 times lower than the OLS polynomial. This is hardly surprising, as the limitations imposed by OLS regression makes it a bad method for predicting on the Runge function. OLS performs linear regression, and when the function we are trying to predict is non-linear, the results are bound to be worse than using methods that can capture and predict non-linear functions. Due to the non-linear activation functions of the neural networks hidden layers, the networks predicted function is not linear, and thus it can better capture the Runge function. There are however downsides to using neural networks for this simple regression, mainly the increased time used for train-

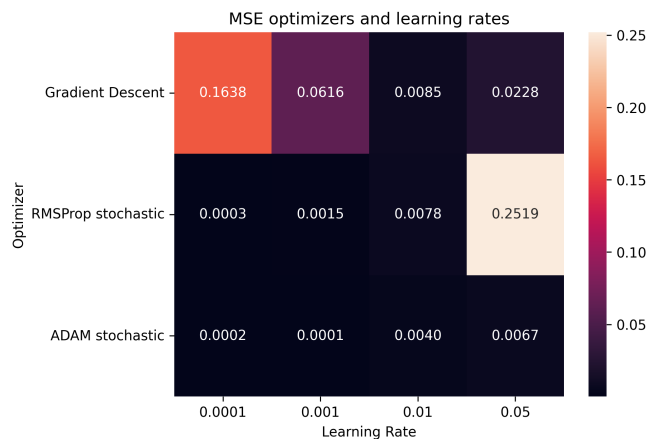


Figure 1. MSE for different optimizers and learning rates

ing the network. OLS regression has a closed-form solution, whereas our best performing network uses 7.04 seconds for training on this simple dataset. For applications where accuracy is not required to be as exact as possible, using OLS regression saves large amounts of computing time for acceptable performance.

Another downside of the neural network approach to regression is the need for large amounts of training data to produce an acceptable result. We can somewhat get around this by using stochastic gradient descent methods, however this might lead us to train the network too long and potentially causing overfitting. Finally, due to the nature of neural networks, the explainability of the resulting model is extremely low, as compared to the polynomial coefficients that regression methods produce.

We have trained the same models with the PyTorch library, to verify the correctness of and to measure the performance of our neural networks. As we can see from table II, our implementation of the neural networks slightly beats the PyTorch network for the simple 1-layer, 50 node architecture. For the more complex model, PyTorch has found a significantly better fit, indicating that there are still areas for improvement in our model. The fact that we have somewhat comparable performance to the existing PyTorch implementations does however suggest that our network is at least capable of achieving the same top-end performance as PyTorch.

To better explore the impact of the learning rate in finding the optimal fit, we explore three different methods to train the network, using plain gradient descent, stochastic RMSProp ($\rho = 0.99$), and stochastic ADAM ($\rho_1 = 0.9$, $\rho_2 = 0.999$). As mentioned before, the training effectiveness relies heavily on the size of the training sample, so we will be expanding to using $n = 1000$ points, using the same setup from NN₂, using Epochs = 500 and a minibatch size of 5.

The plain gradient descent is clearly extremely sensitive to the initial learning rate, as it has no method of adjusting learning rate once it nears convergence. The heatmap in figure 1 clearly shows a trend towards a learn-

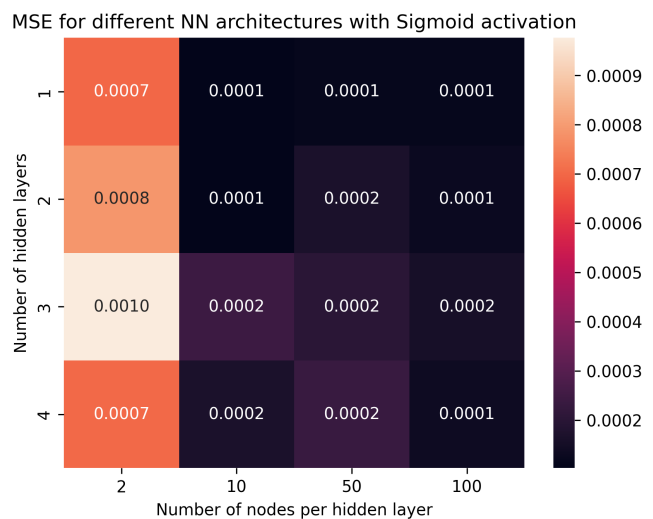


Figure 2. MSE of different network architectures with sigmoid activation function, $\eta = 0.001$

ing rate around 0.01 being optimal for the plain gradient descent. For the adaptive learning rate methods, ADAM outperforms RMSProp in every learning rate, and is significantly more stable and less dependent on the initial learning rate. RMSProp seems to fail to converge well for higher learning rates, which might suggest that RMSProp might benefit from smaller initial learning rates to improve the stability of the algorithm converging. ADAM is in this scenario by far the best alternative as compared to the other methods. As with RMSProp, the lower initial learning rates are better for the convergence of ADAM, as compared to the higher rates required for a good gradient descent result.

We will use the best-performing method, ADAM with learning rate $\eta = 0.001$ for further tests on the effect of activation functions and number of layers and hidden nodes.

While signs of overfitting might be harder to catch in neural networks as compared to regression, there are still some indicators we can look for. Mainly, an overfitted network will show worsening performance with more added complexity, i.e. adding more layers and nodes per layer might make the performance degrade.

Figure 3 shows the signs of overfitting where more complex network architectures perform worse than simpler models. The effect is especially noticeable for larger models with 100 neurons per layer, where for 3 and 4-layer networks, the error increases drastically to a catastrophic level. The sigmoid function, shown in figure 2, shows more stable performance which suggests that it might be more suited for regression purposes than the ReLU function. This might have to do with the fact that the derivative of the sigmoid is a smooth, bounded function, whereas the piecewise ReLU functions are unbounded and might therefore not be as good for regression purposes. The consistent performance of the sigmoid func-

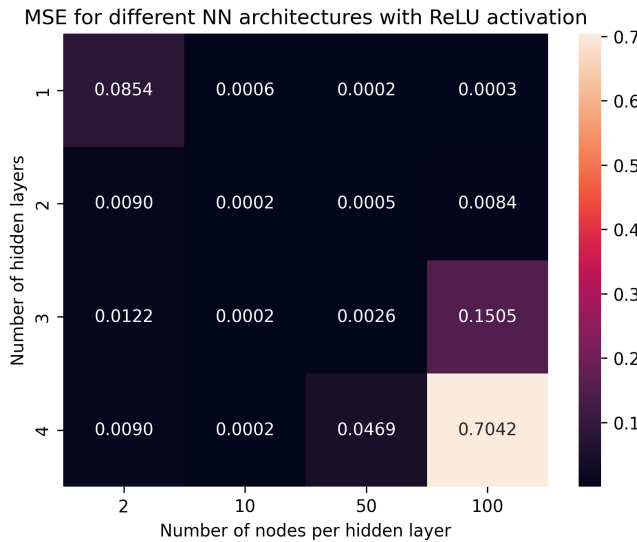


Figure 3. MSE of different network architectures with ReLU activation function

tion, even for extremely small networks (1 layer, 2 nodes) is further evidence of the suitability for regression. ReLU on the other hand struggles with the small network, suggesting that to gain the best performance we need to be much more careful about network architecture when using it as an activation function for regression.

The result that the sigmoid function outperforms the ReLU function to such an extent is not what we would expect given the known problems with the sigmoid function of producing vanishing/exploding gradients. One possibility for why the sigmoid function performs well might be that the range of values output by the sigmoid function ($[0, 1]$) is approximately the same range of values that the training data are scaled to. Therefore, it might be that for the specific case of doing regression on target values in the $[0, 1]$ -range, the sigmoid function might be a better activation function than other alternatives. This also generalizes more broadly, suggesting that activation functions have a large effect on the performance of neural networks.

Both activation functions also show signs of underfitting for extremely small network sizes, in particular 2-node hidden layers are too small for the regression task. For the specific case studied here, a medium-sized network (2-3 layers) with 10-50 node layers might be optimal, and additional complexity only seeming to give diminishing or detrimental effects. The leaky ReLU function was also studied, however due to the similarity to the regular ReLU results, we have decided not to include them, as they provide no additional insight or benefit.

A reason why the performance of more complex architectures starts to degrade might be due to the coefficients becoming too large due to overfitting. One way to combat this is through the use of regularization, particularly L1 and L2 regularizers.

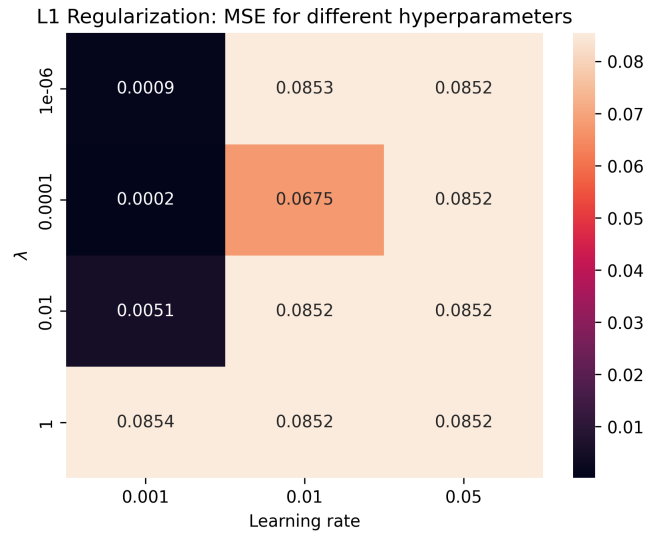


Figure 4. L1 MSE for different learning rates and L1 hyperparameters λ , 4 layer, 100 node layers with ReLU, ADAM SGD

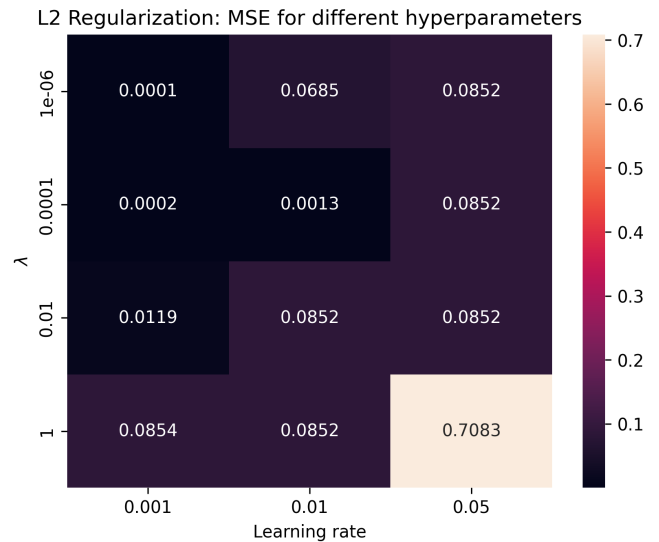


Figure 5. L2 MSE for different learning rates and L2 hyperparameters λ , 4 layer, 100 node layers with ReLU, ADAM SGD

We use the worst performing architecture from the previous test, the 4-layer, 100 node layers with ReLU activation function to see if the regularizers can improve on the 0.7042 MSE from the unregularized model. From the results in figures 4 and 5, it is immediately clear that regularization has a significant effect on the performance of the network. Most configurations of learning rates and regularization parameters have converged to seemingly the same model, both for L1 and L2 regularization, with an MSE of 0.0852. We have not been able to come up with a theory or explanation for this phenomenon, so it is an area that could warrant further study. The $\lambda = 1$,

$\eta = 0.05$ L2 configuration is the only setup that does not improve upon the non-regularized model, this might be due to the combination of a high learning rate making convergence difficult, and too high of a restriction on the weights.

The results seem to favor low learning rates (something we also saw in figure 1), with a low parameter λ , ensuring that the parameters still have a large range of acceptable values, but preventing the problem of weights exploding in magnitude. L1 regularization also seems to produce much more consistent results, although L2 shows much better performance for certain setups. For both methods, low learning rates and hyperparameters λ have the highest effect on our dataset. Due to the scaling and limited range of our data values, we might find that for data with greater fluctuations in magnitude, or for models that have a high risk of being overfitted, that a higher parameter λ could perform better.

For some perspective on the performance of the regularized neural networks, we compare them against normal Ridge and LASSO regression performed on the same dataset. For Ridge, using the parameters $\lambda = 10^{-6}$ and $\eta = 0.001$ using the same stochastic ADAM optimization, the regression yields an MSE of 0.011893, almost 120 times worse than the L2 neural network performed on the same parameters. The results are almost identical for LASSO regression (L1 regularization), using $\lambda = 0.0001$ and $\eta = 0.001$, we get an MSE of 0.011892, almost 60 times worse than the NN.[4] From this it is clear that given the correct setup of hyperparameters and learning rates, neural networks far outperform regular regression methods.

B. MNIST classification

To further test the implemented neural network, we use it for a multiclass classification problem on the MNIST dataset. As mentioned, the input data is flattened to 784-dimensional vectors, which are normalized to the range $[0,1]$ to improve convergence. The labels are one-hot encoded to length 10 vectors. As we have already seen that the network is able to learn at a good rate, we begin the experiments by performing a search through different configurations of neural networks. This consists of picking all combinations of an activation function (sigmoid, ReLU and leaky ReLU), hidden layer architectures ($[30]$, $[100]$, $[200]$, $[100, 50]$, $[200, 200]$), optimizers (SGD, RMSProp, ADAM), and learning rates ($[0.0005, 0.001, 0.005]$ for ADAM / RMSProp, $[0.01, 0.05, 0.1]$ for SGD). We use a fixed setup of Epochs = 15 and a minibatch size of 100. The final layer of each model is given by a linear function, and we use cross entropy as the cost function. Each model is evaluated with the accuracy score on the test set.

Our best performing configuration, using the parameters shown in table III, achieves an accuracy of 96.23% on the test set. Although this is nowhere near optimal,

Parameter	Value
Layers	$[100, 50]$
Activation function	leaky ReLU
Optimizer	SGD
η	0.05
Epochs	15
Minibatch size	100
Regularizer	None

Table III. Parameters for the best performing model (96.23%)

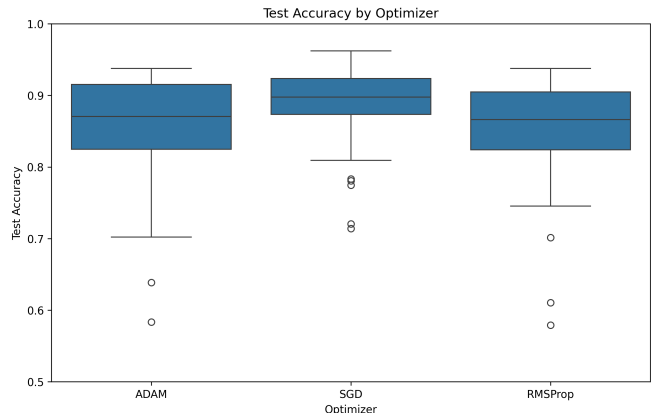


Figure 6. Test accuracy for different optimizers on MNIST

it is an expected outcome given the simple nature of our FFNN.[5]

Figure 6 shows the test accuracy for each optimizer over all configurations. Somewhat interestingly the SGD outperforms both of the more complex optimizers by a wide margin, in contrast with its poor performance on the regression cases. The reason for this is not exactly clear, as we should expect the more complicated models to outperform the simple SGD, however the simplicity that the plain stochastic gradient descent brings might be the reason for it outperforming the others. There are also no configurations that fail to converge at all (such models would have around a 10% accuracy), which might suggest that the dataset and models are simple enough not to cause any problems with vanishing/exploding gradients. Another cause for this difference might also be the higher learning rates given to the SGD, as compared to the adaptive learning rate methods.

Figure 7 shows the reason behind our choices of learning rates. Although the learning rates for SGD are some orders of magnitude larger than for the other optimizers, they both perform similarly. One explanation for the worse performance we see in figure 6 is the heavy tail for the ADAM and RMSProp curves with the learning rate of 0.0005. Both curves also show an upwards trend towards the higher learning rates, as well as improved consistency with the accuracy scores having lower variance. We also see that the ADAM and RMSProp optimizers perform almost identically, with ADAM only slightly better for all the tested learning rates.



Figure 7. Test Accuracy vs Learning Rate by Optimizer on MNIST

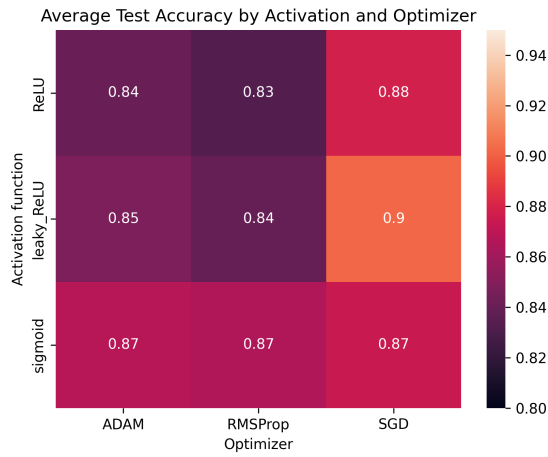


Figure 8. Average Test Accuracy by Activation and Optimizer on MNIST

To investigate the possibility of a link between certain activation functions and optimizers, figure 8 shows average accuracy for combinations of optimizer and activation functions. Again, the SGD results are better than other optimizers, however this is not the case when we use the sigmoid activation function. The exact cause of the sigmoid function not having any preferred optimizer is something that would need further research to explain. Observe also that the combination of the leaky ReLU and the SGD is the most accurate of all combinations, and this is also where we find our best performing model.

The effect that depth and size of each layer has on the networks performance can be seen clearly in figure 9. There is a clear trend towards the more complex models performing better and being more consistent. Some of the outliers for the extremely simple network with one 30-node layer matches that of some complex architectures, however the greater variance in the accuracy of simple models shows that a simple model needs more tuning to obtain good results. There is also a mild tendency of the

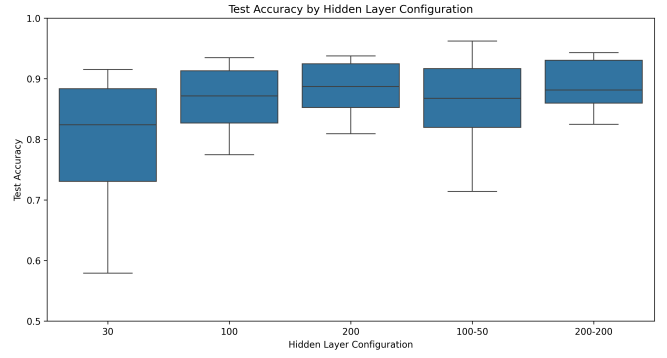


Figure 9. Test Accuracy by Hidden Layer Configuration on MNIST (ReLU activation, ADAM optimizer, 10 epochs).



Figure 10. Training Time by Hidden Layer Configuration on MNIST (ReLU activation, ADAM optimizer, 10 epochs).

models to prefer more nodes in each layer over a deeper network. The slight drop in accuracy and increase in variance of the [100, 50]-network over the [200]-network suggests that it could be beneficial to investigate even more nodes in a few layers rather than a deeper network.

The downside to more complex models can clearly be shown in figure 10. As noted previously, due to the simple nature of the FFNN implementation, the amount of matrix multiplications required for training larger and more complex network architectures becomes so large that a single-threaded program running on the CPU will struggle with performance. The most complex models take upwards of an hour of training time, which makes investigating larger layer sizes and deeper networks challenging and somewhat infeasible. This is also one of the main drawbacks of the neural network approach to classification problems, due to the enormous computational expense that training networks takes. Like for the previous regression case, there are methods like logistic regression that take much less computation to produce results that are somewhat similar.

The confusion matrix, shown in figure 11, gives a deeper insight into areas of improvement for the optimal model. Most symbols are predicted somewhat correctly, however there are some common misclassifications, some examples of which are shown in figure 12. These edge

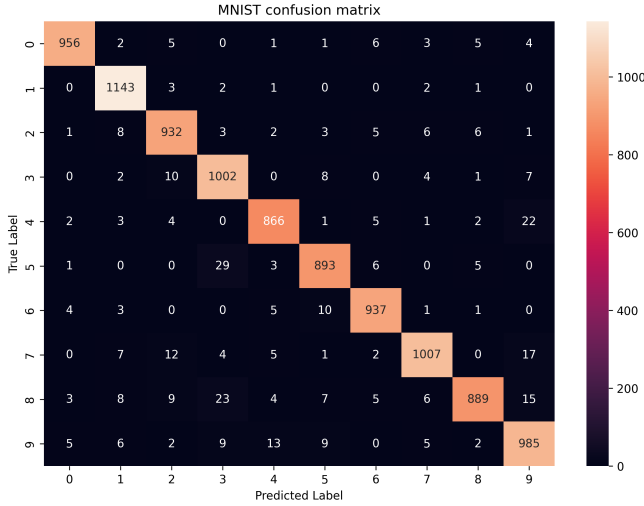


Figure 11. MNIST confusion matrix

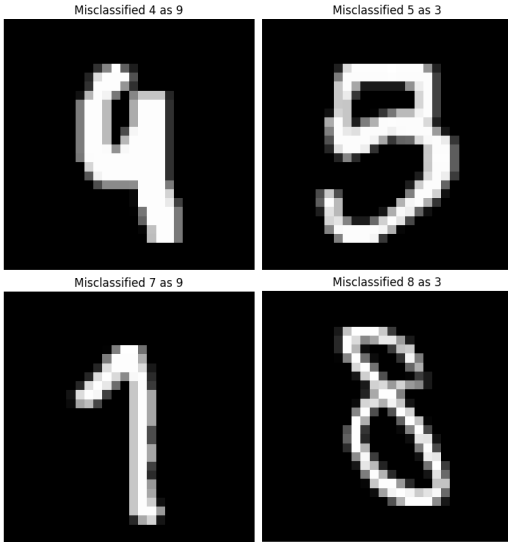


Figure 12. Examples of misclassified digits

cases shows the difficulty in trying to achieve a perfect accuracy score on the MNIST dataset, as many of the digits are somewhat ambiguous and are extremely similar. Although not perfect, there is still room for improvement, as some of the misclassifications shown in figure 12 are still quite clearly not the predicted numbers, in particular the misclassified 8. We can also see from the diagonal that some numbers are more common in the test set than others. This might lead to a somewhat unfair test, as some numbers will be weighted more than others. Ideally, we should have a test set comprised of exactly 1000 digits of each number, so all classes are weighted equally.

For comparison and validation, we have implemented and ran identical tests using the PyTorch implementation of FFNNs. The best performing model using PyTorch beats our best model by a relatively large margin,

Parameter	Value
Layers	[200, 200]
Activation function	leaky ReLU
Optimizer	RMSProp
η	0.0005
Epochs	15
Minibatch size	100
Regularizer	None

Table IV. Parameters for the best performing PyTorch model (98.31%)

achieving a 98.31% accuracy score on the test set. Table IV shows that the configuration for the optimal PyTorch network is slightly different from our implementation. The use of the leaky ReLU function is still the same, however it uses a larger network with the RMSProp optimizer. One other benefit to the PyTorch implementation is the significantly increased performance that the underlying C++ implementation of PyTorch makes training much more efficient. Due to our FFNN implementation performing to a similar level to PyTorch for both regression and classification, we can with some confidence assume that the underlying methods and architecture are implemented correctly, and any performance is only lost due to the more advanced methods and algorithms used by PyTorch.

IV. CONCLUSION

We have demonstrated that our FFNN achieves substantially better performance than traditional OLS regression methods on the Runge function. Even small architectures (few layers and nodes per layer) vastly outperform the linear regression methods previously studied. We do however not observe much benefit of deeper and more complex networks, as there is no discernible benefit of a more complex network, and in some cases performance decreases. We observed signs of overfitting for complex networks using the ReLU activation function, however these signs were not present for the same architecture using the sigmoid function. This suggests that the sigmoid function might be more suited for this particular application. For the training of regression NNs, we observe that a stochastic ADAM gradient descent is by far the most stable and performant of the algorithms tested. Applying L1/L2 regularization to our worst-performing unregularized network architecture drastically improves results for almost all configurations of learning rates and hyperparameters tested. We do however observe that most configurations for both regularization methods converge to the exact same MSE, the explanation for this warrants further study. The study of other functions for regression is also important, as it is important to understand whether the trends we see in regression on the Runge function generalizes.

On classification, the FFNN performs with a high ac-

curacy of up to 97% while using a relatively simple network architecture. The amount of work to obtain such high performance is however quite large, as the tuning of parameters and network architecture has a much larger role in the accuracy of the model. The stochastic ADAM gradient descent optimizer is again the most stable and best performing method for training the network, suggesting that it is a good general-purpose optimizer performing well on both classification and regression cases.

In a comparison against existing PyTorch implementations of the FFNN and optimization algorithms, we see that our network has the same capability to perform at an equal level given the right tuning of parameters. Our network is however much more unstable when it comes to the average-case performance, indicating there is room for improvement in the stability of the optimizers implemented. Due to the performance limitations posed by the simple implementations, we are not able to test for more complex networks, which may have better results.

-
- [1] C. Runge *et al.*, Über empirische funktionen und die interpolation zwischen äquidistanten ordinaten, *Zeitschrift für Mathematik und Physik* **46**, 20 (1901).
 - [2] L. Deng, The mnist database of handwritten digit images for machine learning research, *IEEE Signal Processing Magazine* **29**, 141 (2012).
 - [3] A. L. Maas, Rectifier nonlinearities improve neural network acoustic models (2013).
 - [4] S. Oppen and O. M. Bråthen, Linear regression and resampling techniques on the runge function, (2025).
 - [5] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, Gradient-based learning applied to document recognition, *Proceedings of the IEEE* **86**, 2278 (1998).